

TLS

Traffic Light Simulation

Dokumentasi Lengkap & Analisis Performa

Versi 1.0.0

Engine: SUMO 1.27 + TraCI

GUI: PyQt6 · Chart: pyqtgraph

Bahasa: Python 3.10–3.13

Platform: Linux / Windows / macOS

Daftar Isi

1. [Gambaran Umum Proyek](#)
2. [Tech Stack & Istilah Penting](#)
3. [Struktur Folder \(Project Tree\)](#)
4. [Panduan Memulai \(Quick Start\)](#)
5. [Tutorial Lengkap Git](#)
6. [Arsitektur Kode \(MVC\)](#)
7. [Analisis Performa & Bottleneck](#)
8. [Prioritas Perbaikan & Penanggung Jawab](#)
9. [Cara Build & Deploy](#)
10. [Strategi Testing](#)
11. [Glosarium Istilah](#)
12. [Lampiran: Referensi File Penting](#)

1. Gambaran Umum Proyek

1.1 Apa Itu TLS?

TLS (Traffic Light Simulation) adalah aplikasi desktop untuk simulasi lalu lintas skala kota. Aplikasi ini menggunakan **SUMO** (Simulation of Urban MObility) sebagai engine simulasi, dan menyediakan GUI interaktif real-time untuk mengatur, memantau, dan menganalisis lampu lalu lintas (TL).

1.2 Fitur Unggulan

- **3 map preset:** Pamulang (Indonesia), Silicon Valley (USA), Tokyo (Japan) — masing-masing dengan TL dan kendaraan realistis
- **4 algoritma TL:** Fixed-Time, Actuated, Green Wave, Max-Pressure — bisa dipilih dan dibandingkan
- **Dashboard real-time:** Grafik kecepatan rata-rata, waktu tunggu, throughput, panjang antrian, konsumsi BBM, emisi CO₂
- **Import OSM:** Download peta dari OpenStreetMap → langsung bisa disimulasi
- **Ekspor CSV/JSON:** Data simulasi bisa diekspor untuk analisis lanjutan
- **Multi-platform:** Linux (Ubuntu), Windows (10/11), macOS

1.3 Latar Belakang & Masalah

Lampu lalu lintas di banyak kota masih menggunakan pengaturan waktu statis (fixed-time). Hal ini menyebabkan:

- Kemacetan di jam sibuk karena TL tidak bisa menyesuaikan dengan volume kendaraan
- Pemborosan bahan bakar & polusi dari kendaraan yang berhenti terlalu lama
- Waktu tempuh yang tidak stabil

TLS memungkinkan peneliti dan perencana kota untuk menguji algoritma TL adaptif (Actuated, Max-Pressure, Green Wave) pada skenario lalu lintas nyata atau sintetis sebelum diimplementasikan di lapangan.

1.4 Siapa Saja yang Terlibat?

Peran	Tanggung Jawab	Bagian Kode
Backend Engineer	TraCI client, koneksi SUMO, caching, subscription, thread safety	<code>traci_client.py</code> , <code>sim_controller.py</code> (bagian TraCI)
Algorithm Engineer	Fixed-Time, Actuated, Max-Pressure, Green Wave logic	<code>tl_algorithms.py</code> , <code>traffic_light.py</code>
Frontend / GUI Engineer	Map rendering, dashboard, kontrol panel, user experience	<code>map_viewer.py</code> , <code>dashboard.py</code> , <code>main_window.py</code>
DevOps / Build Engineer	PyInstaller build, GitHub Actions CI/CD, release management	<code>tls.spec</code> , <code>.github/workflows/build.yml</code>
QA / Tester	Unit test, integration test, performance benchmark, regression	<code>tests/</code> (akan dibuat)

2. Tech Stack & Istilah Penting

2.1 Teknologi yang Digunakan

Komponen	Teknologi	Versi	Fungsi
Simulation Engine	SUMO	1.20+ (1.27)	Menjalankan simulasi lalu lintas (kendaraan, jalan, TL)
Komunikasi	TraCI	bawaan SUMO	Protokol TCP untuk kontrol real-time dari Python ke SUMO
GUI Desktop	PyQt6	>= 6.5	Window, map, kontrol, menu — semua tampilan aplikasi
Grafik Real-time	pyqtgraph	>= 0.13	Chart kecepatan, throughput, dll di dashboard
PDF (opsional)	weasyprint	>= 60	Export laporan ke PDF
Bahasa	Python	3.10–3.13	Semua kode aplikasi
Database	SQLite	bawaan Python	Menyimpan hasil simulasi & skenario

2.2 Glosarium Istilah Penting

Istilah	Kepanjangan	Penjelasan
SUMO	Simulation of Urban MObility	Engine simulasi lalu lintas open-source dari DLR (Jerman). Bisa simulasi seluruh kota dengan ribuan kendaraan.
TraCI	Traffic Control Interface	Protokol TCP yang menghubungkan Python ke SUMO. Semua perintah (get posisi kendaraan, set fase TL, dll) lewat TraCI.
TL	Traffic Light	Lampu lalu lintas. Di TLS, setiap TL punya beberapa fase (hijau, kuning, merah) dengan durasi tertentu.
Fase TL	TL Phase	Kondisi lampu pada suatu waktu, misal: "ggggrrrr" = 4 lajur hijau + 4 lajur merah.
Fixed-Time	—	Algoritma TL paling sederhana: setiap fase punya durasi tetap, berputar terus.
Actuated	—	Algoritma yang memperpanjang fase hijau jika ada kendaraan mendekat (via detector/induction loop).
Max-Pressure	—	Algoritma yang memilih fase TL berdasarkan jumlah kendaraan di ruas jalan terpadat. Paling kompleks & adaptif.
Green Wave	—	Algoritma yang menyelaraskan TL secara berurutan agar kendaraan bisa melewati banyak TL tanpa berhenti.
Subscription	—	Fitur TraCI: kita bilang ke SUMO "tolong kirim data X setiap step" sekali, lalu tinggal baca hasilnya. Lebih cepat daripada panggil perintah setiap step.
Step / Simulation Step	—	Satu iterasi simulasi (default 1 detik). SUMO menghitung posisi kendaraan baru, TL berpindah fase, dll.
Thread / Threading	—	Eksekusi paralel. Simulasi jalan di thread sendiri, GUI di thread utama. Kalau akses data barengan tanpa pengaman → crash (race condition).
Race Condition	—	Dua thread mengakses data yang sama pada saat bersamaan, menyebabkan hasil tidak terduga / crash.
PyInstaller	—	Tools untuk mengemas aplikasi Python jadi executable (.exe di Windows, binary di Linux).
GitHub Actions	—	Layanan CI/CD dari GitHub: otomatis menjalankan build & test saat kita push kode.

2.3 Kenapa Python 3.14 Tidak Didukung?

Python 3.14 masih terlalu baru. Library **PyQt6** dan **pyqtgraph** belum merilis wheel (pre-built binary) untuk Python 3.14. Akibatnya, pip akan mencoba kompilasi dari source code, yang biasanya gagal di Windows dan memakan waktu lama di Linux. Gunakan Python 3.12 atau 3.13.

3. Struktur Folder (Project Tree)

Proyek ini mengikuti pola **MVC** (Model-View-Controller) — dijelaskan lebih lanjut di Bab 6.

```
traffic-light-sim/
|-- app/                                # KODE UTAMA APLIKASI
|   |-- main.py                         # Entry point: dijalankan dengan "python -m app.main"
|   |
|   |-- engine/                         # LAYER MODEL + CONTROLLER
|   |   |-- traci_client.py             # TraCI wrapper (Model) - 499 baris
|   |   |-- sim_controller.py           # Loop simulasi (Controller) - 298 baris
|   |   |-- tl_algorithms.py            # 4 algoritma TL - 173 baris
|   |   |-- osm_importer.py            # Import OSM -> netconvert - ~80 baris
|   |
|   |-- gui/                            # LAYER VIEW (tampilan)
|   |   |-- map_viewer.py               # Map + kendaraan + TL (QGraphicsView) - 674 baris
|   |   |-- main_window.py              # Layout jendela, menu, signal - 233 baris
|   |   |-- dashboard.py                # Chart real-time (pyqtgraph) - 181 baris
|   |   |-- controls.py                 # Toolbar Play/Pause/Speed - ~150 baris
|   |   |-- config_panel.py             # Panel konfigurasi algoritma
|   |   |-- settings_dialog.py          # Dialog pengaturan aplikasi
|   |   |-- scenario_dialog.py          # Dialog simpan/load skenario
|   |   |-- tile_provider.py            # Background peta dari OSM tiles
|   |
|   |-- models/                         # DATA CLASSES
|   |   |-- traffic_light.py            # TLPhase, TrafficLight
|   |   |-- vehicle.py                 # Vehicle (posisi, kecepatan, sudut)
|   |
|   |-- metrics/                        # PENGUMPUL DATA
|   |   |-- collector.py                # MetricsCollector (ring buffer)
|   |   |-- storage.py                  # Simpan ke SQLite / JSON
|   |
|   |-- utils/                          # UTILITAS
|   |   |-- config.py                   # Baca/tulis konfigurasi (JSON)
|   |   |-- localization.py             # Bahasa Indonesia / Inggris
|   |   |-- logger.py                   # Logging (loguru)
|   |
|-- sim/                                # DATA SIMULASI (map per folder)
|   |-- pamulang/                       # Map Pamulang, Indonesia
|   |-- silicon_valley/                 # Map Silicon Valley, USA
|   |-- tokyo/                          # Map Tokyo, Japan
|   |
|-- scripts/                            # SCRIPT UTILITAS
|   |-- setup_maps.py                   # Inisialisasi folder map
|   |-- generate_logo.py                 # Generate icon.png + icon.ico
|   |-- generate_docs_pdf.py            # PEMBUAT PDF INI
|   |-- add_tls_to_network.py           # Tambah TL ke jaringan
|   |
|-- tests/                              # (AKAN DIBUAT - lihat Bab 10)
|   |
|-- .github/workflows/                  # CI/CD pipeline (GitHub Actions)
|   |-- build.yml
|   |
|-- setup.bat                           # Setup untuk Windows (CMD)
|-- setup.ps1                           # Setup untuk Windows (PowerShell)
|-- setup.sh                             # Setup untuk Linux / macOS
|-- tls.spec                             # Konfigurasi PyInstaller
|-- requirements.txt                     # Daftar dependency Python
|-- README.md                           # Dokumentasi cepat
|-- TLS-Dokumentasi.pdf                  # DOKUMEN INI
```

4. Panduan Memulai (Quick Start)

4.1 Prasyarat

1. **Python 3.10, 3.11, 3.12, atau 3.13** — Install dari python.org. Centang "Add Python to PATH" saat instalasi.
2. **SUMO 1.20 atau lebih baru** — Download dari sumo.dlr.de. Pastikan folder `bin/` terdaftar di PATH environment variable, atau set `SUMO_HOME` ke folder instalasi.

PERINGATAN: Python 3.14 TIDAK didukung. PyQt6 dan pyqtgraph belum punya wheel untuk 3.14.

4.2 Cara Clone & Setup

Linux / macOS

```
git clone https://github.com/Cefneal/traffic-light-sim.git
cd traffic-light-sim
bash setup.sh
source venv/bin/activate
python -m app.main
```

Windows (PowerShell — disarankan)

```
git clone https://github.com/Cefneal/traffic-light-sim.git
cd traffic-light-sim
.\setup.ps1
.\venv\Scripts\Activate.ps1 ; python -m app.main
```

Windows (CMD)

```
git clone https://github.com/Cefneal/traffic-light-sim.git
cd traffic-light-sim
setup.bat
venv\Scripts\activate & python -m app.main
```

4.3 Penjelasan Langkah-langkah

1. **git clone** — Mendownload semua kode dari GitHub ke komputer lokal
2. **cd** — Masuk ke folder proyek
3. **setup.bat / setup.ps1 / setup.sh** — Script yang otomatis: bikin virtual environment (venv), install semua dependency Python (PyQt6, pyqtgraph, traci, weasyprint)
4. **Activate venv** — Mengaktifkan lingkungan Python terisolasi (biar library yang diinstall gak bentrok dengan Python sistem)
5. **python -m app.main** — Menjalankan aplikasi

4.4 Cara Pakai Aplikasi

1. Pilih map dari dropdown (Pamulang / Silicon Valley / Tokyo)
2. Atur algoritma TL di panel Configuration (Fixed-Time / Actuated / Green Wave / Max-Pressure)
3. Klik tombol **Play** (▶) — simulasi berjalan
4. Lihat grafik real-time di panel Dashboard (kanan)
5. Atur kecepatan simulasi dengan slider Speed
6. Export data via File → Export CSV atau Export JSON

5. Tutorial Lengkap Git

5.1 Apa Itu Git?

Git adalah sistem version control. Git mencatat setiap perubahan kode — siapa yang mengubah, kapan, dan apa yang diubah. Ini penting untuk:

- Kerja tim — beberapa orang bisa mengedit kode yang sama tanpa konflik
- Riwayat — bisa lihat/bandingkan versi kode sebelumnya
- Rollback — kalau ada error, bisa kembali ke versi yang stabil
- Branching — mengembangkan fitur baru tanpa mengganggu kode utama

5.2 Install Git

OS	Perintah
Linux (Ubuntu/Debian)	<code>sudo apt install git</code>
macOS	<code>brew install git</code>
Windows	Download dari git-scm.com — pastikan centang "Git Bash" dan "Add to PATH"

5.3 Konfigurasi Awal (Hanya Sekali)

```
git config --global user.name "Nama Kamu"
git config --global user.email "email@example.com"
git config --global init.defaultBranch main
```

5.4 Clone Repository (Download Pertama Kali)

```
git clone https://github.com/Cefneal/traffic-light-sim.git
cd traffic-light-sim
```

Penjelasan:

- `git clone` — Download seluruh riwayat proyek dari GitHub ke folder lokal
- Setelah clone, folder `traffic-light-sim/` akan muncul dengan semua file
- Git otomatis membuat remote bernama `origin` yang mengarah ke GitHub

5.5 Status & Log (Cek Kondisi)

```
# Cek file yang berubah
git status

# Lihat riwayat commit
git log --oneline --graph --all

# Bandingkan perubahan yang belum di-commit
git diff

# Lihat detail commit terakhir
git show
```

5.6 Pull (Ambil Perubahan Terbaru)

```
git pull origin main
```

Sebelum mulai bekerja, selalu `git pull` dulu untuk mendapatkan perubahan terbaru dari GitHub.

5.7 Branch (Cabang)

Branch memungkinkan kita mengembangkan fitur secara terpisah tanpa mengganggu kode utama (`main`).

```
# Lihat daftar branch
git branch -a

# Buat branch baru
git checkout -b feature/perbaikan-kinerja

# Pindah ke branch lain
```

```
git checkout main

# Hapus branch (sudah di-merge)
git branch -d feature/perbaikan-kinerja
```

Diagram alur branch:

```
main:    A --- B --- C --- D --- E
         \       /
feature:  X --- Y
```

Keterangan: `feature` bercabang dari commit B, membuat commit X dan Y, lalu di-merge kembali ke main di commit D.

5.8 Add, Commit, Push (Siklus Harian)

Setiap kali selesai mengerjakan bagian kecil, commit.

```
# 1. Cek apa saja yang berubah
git status

# 2. Stage file yang ingin di-commit
git add app/engine/sim_controller.py
git add app/engine/traci_client.py

# Atau stage semua file sekaligus (hati-hati)
git add -A

# 3. Commit dengan pesan yang jelas
git commit -m "fix: kurangi TraCI calls di simulation loop"

# 4. Push ke GitHub
git push origin nama-branch-kamu
```

5.9 Contoh Skenario Lengkap

```
# Pagi hari: pull dulu
git checkout main
git pull origin main

# Buat branch untuk fitur baru
git checkout -b fix/fuel-loop

# Edit file traci_client.py
# ... (perbaiki kode) ...

git add app/engine/traci_client.py
git commit -m "fix: ganti fuel loop dengan subscription"

# Push ke GitHub
git push -u origin fix/fuel-loop

# Di GitHub: buat Pull Request (PR)
# Minta review dari tim
# Setelah disetujui: merge ke main

# Hapus branch lokal
git checkout main
git pull origin main
git branch -d fix/fuel-loop
```

5.10 Pull Request (PR)

Pull Request adalah permintaan untuk menggabungkan kode dari branch kamu ke branch utama. Langkah-langkah:

1. Push branch kamu ke GitHub (lihat 5.9)
2. Buka repo di GitHub.com → klik "Compare & pull request"
3. Tulis judul dan deskripsi perubahan
4. Klik "Create pull request"
5. Tim akan review, memberi komentar, atau menyetujui
6. Setelah disetujui, klik "Merge pull request"

5.11 Aturan Penulisan Commit

Prefix	Arti	Contoh
<code>fix:</code>	Perbaikan bug	<code>fix: race condition di thread TraCI</code>
<code>feat:</code>	Fitur baru	<code>feat: tambah dialog import OSM</code>

perf:	Optimasi kinerja	perf: cache hasil subscription kendaraan
docs:	Perubahan dokumentasi	docs: update README setup Windows
refactor:	Perubahan kode (tanpa ubah fitur)	refactor: pisahkan logika TL building
chore:	Maintenance	chore: update requirements.txt

5.12 Tag & Release

Tag menandai versi tertentu di riwayat Git. Tag memicu build otomatis di GitHub Actions.

```
# Buat tag
git tag -a v1.0.0 -m "Release v1.0.0"

# Push tag ke GitHub
git push origin v1.0.0

# Lihat daftar tag
git tag -l

# Hapus tag (jika salah)
git tag -d v1.0.0
git push origin :refs/tags/v1.0.0
```

Setelah tag di-push, GitHub Actions akan:

1. Menjalankan semua test
2. Build executable Linux + Windows
3. Upload ke halaman Releases

5.13 Mengatasi Konflik Merge

Konflik terjadi ketika dua orang mengubah baris yang sama di file yang sama. Git tidak tahu mana yang benar → kita harus memilih secara manual.

```
# Saat merge, Git akan bilang: "CONFLICT in file.txt"
# Buka file tersebut. Cari tanda ini:
# <<<<<< HEAD
# kode dari branch main
# =====
# kode dari branch kamu
# >>>>>> nama-branch

# Hapus tanda <<< === >>>, sisakan kode yang benar
# Lalu:
git add file.txt
git commit -m "merge: resolve conflict di file.txt"
```

5.14 Git Stash (Simpan Sementara)

Kalau lagi ngerjain sesuatu tapi butuh pindah branch dulu:

```
# Simpan perubahan sementara
git stash

# Pindah branch, pull, dll
git checkout main
git pull

# Kembali kerja
git checkout feature/*
git stash pop
```

5.15 Git Rebase (Alternatif Merge yang Lebih Rapi)

`rebase` mengambil commit dari branch kita dan menempelkannya di ujung branch main. Hasilnya riwayat lebih linear & bersih.

```
git checkout feature/*
git rebase main
# Jika ada konflik, selesaikan, lalu:
git rebase --continue
# Kalau bingung / mau batal:
git rebase --abort
```

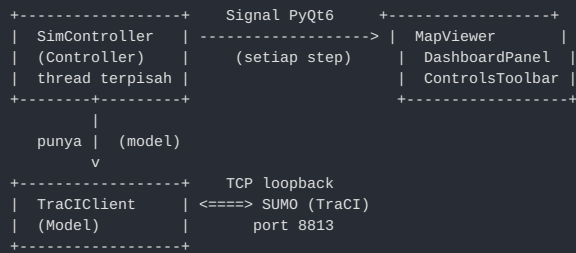
Catatan: Jangan `rebase` di branch yang sudah dipush dan digunakan orang lain. Rebase hanya untuk branch lokal.

6. Arsitektur Kode (MVC)

6.1 Pola MVC (Model-View-Controller)

TLS menggunakan pola arsitektur **MVC** — memisahkan data (Model), logika (Controller), dan tampilan (View):

- **Model** (`TraCIClient`) — Bertanggung jawab atas semua komunikasi dengan SUMO via TraCI. Menyediakan data mentah (posisi kendaraan, status TL, dll).
- **Controller** (`SimController`) — Mengatur alur simulasi. Setiap step: majukan SUMO, jalankan algoritma TL, kumpulkan metrik, kirim data ke View.
- **View** (`MapView` , `DashboardPanel` , dll) — Menampilkan data ke pengguna. Tidak boleh langsung bertanya ke SUMO — harus lewat Controller.



6.2 Aliran Data Setiap Step

Simulasi berjalan di thread terpisah (~30 step/detik). GUI membaca data dari subscription (tanpa panggilan TraCI tambahan).

Urutan	Thread	Aksi	Panggilan TraCI
1	Sim	<code>get_vehicle_ids()</code> → subscribe vehicle baru	2
2	Sim	<code>simulationStep()</code> → SUMO maju 1 step	1
3	Sim	Baca data kendaraan dari subscription (cached)	0
4	Sim	[BOTTLENECK] Hitung total fuel/CO2 → loop semua kendaraan	$N \times 2$
5	Sim	Jalankan algoritma TL → baca data edge, set phase	10–1500
6	Sim	Kirim signal 'step' ke GUI (via PyQt6)	0
7	GUI	Baca posisi kendaraan dari subscription (cached)	0
8	GUI	Update TL state dari subscription	0

6.3 TraCIClient (Model)

File: `app/engine/traci_client.py` (499 baris).

Fungsi utama:

- `connect()` — Membuka koneksi TCP ke SUMO di port 8813
- `simulation_step()` — Menyuruh SUMO maju satu step
- `subscribe_*` — Berlangganan data (edge, vehicle, TL) agar bisa dibaca tanpa panggilan TraCI tambahan
- `get_*_cached()` — Membaca data dari hasil subscription (cepat, 0 panggilan TraCI)
- `set_tl_phase()` — Mengubah fase lampu lalu lintas di SUMO

Masalah yang diketahui (lihat Bab 7):

- **Thread-unsafe**: Semua method akses `traci.*` langsung tanpa lock → crash kalau GUI dan Sim akses barengan
- **Fuel/CO2 loop**: `get_total_fuel_consumption()` memanggil `getFuelConsumption()` per kendaraan → ribuan panggilan TraCI per step
- **Subscription leak**: Vehicle di-subscribe tapi tidak pernah di-unsubscribe

6.4 SimController (Controller)

File: `app/engine/sim_controller.py` (298 baris).

Fungsi utama:

- `start()` — Menjalankan SUMO sebagai proses terpisah, konek via TraCI, mulai thread simulasi

- **_run_loop()** — Loop utama: subscribe vehicle → simulationStep() → baca data → jalankan algoritma TL → kumpulkan metrik → emit signal
- **pause() / resume() / stop()** — Kontrol simulasi
- **set_algorithm()** — Ganti algoritma TL (Fixed / Actuated / Max-Pressure / Green Wave)

6.5 GUI (View)

Tiga widget utama:

- **MapView** (674 baris): QGraphicsView dengan QTimer ~30 FPS. Menggambar jalan, kendaraan, TL, jejak kendaraan, heatmap, tile OSM.
- **DashboardPanel** (181 baris): Chart rolling time-series (kecepatan, waktu tunggu, throughput, antrian, BBM, CO2).
- **ControlsToolbar** (~150 baris): Tombol Play/Pause/Stop, slider Speed, dropdown scenario.

7. Analisis Performa & Bottleneck

7.1 Ringkasan

Aplikasi saat ini mengalami **lag parah** karena terlalu banyak panggilan TraCI (protokol TCP). Setiap panggilan TraCI memakan waktu ~100 mikrodetik. Dengan **14.000+ panggilan per step** dan target 30 FPS (30 step/detik), aplikasi menghabiskan sebagian besar waktu menunggu jawaban dari SUMO, bukan menghitung.

Metrik	Saat Ini	Target Setelah Fix
Panggilan TraCI per step	~14.000+	~50–200
FPS GUI (1000 kendaraan)	8–12 (adaptive, turun)	30 (stabil)
Thread safety	Rawan crash (race condition)	Aman (shared buffer + lock)
Memory (simulasi 1 jam)	Terus bertambah (sub leak)	Stabil
Akurasi timing real-time	Melenceng (sleep tanpa koreksi)	< 1% error

7.2 P1 — Critical (Harus Diperbaiki Segera)

Tiga masalah ini menyebabkan lag paling parah dan crash. **Prioritas tertinggi.**

#	Masalah	File:Baris	Dampak	Panggilan/Step
1.1	Loop Fuel/CO ₂ per kendaraan	traci_client.py:401–423	Setiap step, looping semua kendaraan (5000+) untuk baca fuel & CO2 masing-masing	~10.000
1.2	MaxPressure panggil edge berulang	tl_algorithms.py:84–116	Setiap TL, setiap fase, looping semua edge → panggil getLastStepVehicleNumber() + getLastStepMeanSpeed()	~1.500
1.3	Race condition thread (GUI + Sim)	sim_controller.py + map_viewer.py	Dua thread akses TraCI bersamaan → crash "setPhase failed: Connection already closed"	N/A (crash)

Detail P1.1 — Fuel/CO₂ Loop

```
# traci_client.py:401-423
def get_total_fuel_consumption(self) -> float:
    total = 0.0
    for vid in traci.vehicle.getIDList():
        # 5000 ID
        total += traci.vehicle.getFuelConsumption(vid) # 5000 panggilan TraCI
    return total

# Dipanggil SETIAP STEP di sim_controller.py:208-209
total_fuel = self.traci.get_total_fuel_consumption() # 5000 panggilan
total_co2 = self.traci.get_total_co2_emission() # 5000 panggilan
```

FIX: Tambahkan VAR_FUELCONSUMPTION dan VAR_CO2EMISSION ke daftar subscription kendaraan. Dengan cara ini, data fuel & CO2 otomatis tersedia setelah setiap step tanpa panggilan TraCI tambahan. Alternatif: hitung fuel/CO2 hanya setiap 10 step (karena perubahannya lambat). **Effort: ~4 jam. Penanggung jawab: Backend Engineer.**

Detail P1.2 — MaxPressure Panggil Edge Berulang

```
# tl_algorithms.py:96-108
edge_ids = traci_module.edge.getIDList()[1:50] # 1 panggilan
for i, phase in enumerate(tl.phases):
    # ~3 fase
    for eid in edge_ids:
        # ~50 edge
        count = traci_module.edge.getLastStepVehicleNumber(eid) # 1 panggilan
        speed = traci_module.edge.getLastStepMeanSpeed(eid) # 1 panggilan
# Total per TL: 1 + 3 x 50 x 2 = 301 panggilan
```

Dengan ~10 TL, MaxPressure saja menghabiskan ~3.000 panggilan/step.

FIX: Gunakan metode `get_edge_data_cached(eid)` yang sudah ada — ia membaca data dari subscription alih-alih membuat panggilan TraCI baru. Data edge sudah di-subscribe di `sim_controller.py:164-165`. **Effort: ~3 jam. Penanggung jawab: Algorithm Engineer.**

Detail P1.3 — Race Condition Thread

```
# THREAD 1 (Sim): sim_controller.py:195
self.traci.simulation_step() # TraCI: perintah SimulationStep (sibuk)
```

```
# THREAD 2 (GUI): map_viewer.py:406 (dijalankan ~30 Hz)
vehicles = tc.get_all_vehicles_cached() # TraCI: getIDList (samaan!)

# Hasilnya:
# "setPhase failed: Connection already closed"
# GUI freeze / crash acak
```

FIX: Implementasi **shared buffer** yang aman untuk thread. Sim thread menulis data kendaraan & TL ke buffer yang dilindungi `threading.Lock`. GUI thread membaca dari buffer ini. **DILARANG** keras memanggil `traci.*` dari GUI thread. **Effort: ~8 jam. Penanggung jawab: Backend Engineer + GUI Engineer (kolaborasi).**

7.3 P2 — High (Harus Diperbaiki)

#	Masalah	File:Baris	Dampak	Panggilan/Step
2.1	Subscription kendaraan bocor	sim_controller.py:189–192	SUMO makin lambat seiring waktu karena ribuan subscription menumpuk	N/A
2.2	Panggil <code>getIDList()</code> 2x redundant	sim_controller + traci_client	<code>getIDList()</code> dipanggil dua kali per step	~2
2.3	Timing sleep tidak akurat	sim_controller.py:245–246	Simulasi makin lambat vs real-time seiring waktu	0
2.4	<code>step_single()</code> kurang parameter <code>step_length</code>	sim_controller.py:259	Algoritma actuated tidak berfungsi di single-step mode	0

FIX 2.1: Setelah setiap step, hitung selisih: kendaraan yang sudah keluar = `subscribed - current_ids`. Unsubscribe kendaraan yang sudah pergi via `traci.vehicle.unsubscribe(vid)`. **Effort: ~3 jam. Backend Engineer.**

FIX 2.2: Cache vehicle IDs dari hasil subscription — jangan panggil `getIDList()` dua kali. **Effort: ~1 jam. Backend Engineer.**

FIX 2.3: Ganti `time.sleep(sleep_time)` dengan `actual_sleep = max(0, target_interval - elapsed_body_time)`. **Effort: ~2 jam. Backend Engineer.**

FIX 2.4: Tambahkan parameter `step_length` ke pemanggilan `self._algorithm_fn(tl, traci, self.current_time, step_length)`. **Effort: ~1 jam. Backend Engineer.**

7.4 P3 — Medium (Perbaikan Tambahan)

#	Masalah	File:Baris	Dampak
3.1	TL <code>get_tl_ids()</code> dipanggil 2x per frame GUI	map_viewer.py:526,543	Panggilan TraCI ganda per update
3.2	Dashboard update 30 Hz (boros CPU)	main_window.py:114–128	Chart re-render 30/detik padahal tidak perlu
3.3	Cleanup kendaraan $O(n)$ scan semua dict	map_viewer.py:508–519	Iterasi semua item setiap frame
3.4	Bug <code>p.index</code> vs <code>p.next</code> di <code>get_tl_program()</code>	traci_client.py:301	Index fase tersimpan salah

FIX 3.1: Cache `tl_ids` setelah lazy init, jangan panggil `get_tl_ids()` ulang. **Effort: ~0.5 jam. GUI Engineer.**

FIX 3.2: Throttle update dashboard ke 4–5 Hz (gunakan counter atau QTimer terpisah). **Effort: ~1 jam. GUI Engineer.**

FIX 3.3: Gunakan set difference: `gone = old_set - new_set`. **Effort: ~1 jam. GUI Engineer.**

FIX 3.4: Ganti `p.next` jadi `p.index`. **Effort: ~0.5 jam. Backend Engineer.**

8. Prioritas Perbaikan & Penanggung Jawab

8.1 Sprint 1: P1 — Critical (Estimasi 2–3 hari)

#	Tugas	Estimasi	File yang Diubah	Penanggung Jawab
1.1	Hapus loop Fuel/CO2; baca dari subscription atau hitung tiap 10 step	4 jam	traci_client.py, sim_controller.py	Backend Engineer
1.2	Ubah max_pressure pakai get_edge_data_cached()	3 jam	tl_algorithms.py	Algorithm Engineer
1.3	Bikin shared buffer thread-safe; pisahkan akses TraCI dari GUI	8 jam	traci_client.py, map_viewer.py, sim_controller.py	Backend + GUI Engineer

8.2 Sprint 2: P2 — High (Estimasi 2 hari)

#	Tugas	Estimasi	File yang Diubah	Penanggung Jawab
2.1	Unsubscribe kendaraan yang sudah keluar jaringan	3 jam	traci_client.py, sim_controller.py	Backend Engineer
2.2	Cache vehicle ID biar gak dobel panggil getIDList()	1 jam	traci_client.py	Backend Engineer
2.3	Perbaiki timing: actual_sleep = target - elapsed_body	2 jam	sim_controller.py	Backend Engineer
2.4	Tambah step_length ke pemanggilan step_single()	1 jam	sim_controller.py	Backend Engineer

8.3 Sprint 3: P3 — Medium (Estimasi 1 hari)

#	Tugas	Estimasi	File yang Diubah	Penanggung Jawab
3.1	Cache tl_ids setelah lazy init di map_viewer	0.5 jam	map_viewer.py	GUI Engineer
3.2	Throttle dashboard ke 4–5 Hz	1 jam	main_window.py	GUI Engineer
3.3	Pakai set difference untuk cleanup kendaraan	1 jam	map_viewer.py	GUI Engineer
3.4	Fix p.next → p.index di get_tl_program()	0.5 jam	traci_client.py	Backend Engineer

8.4 Ringkasan Penanggung Jawab

Peran	Nama (contoh)	Tanggung Jawab (P1/P2/P3)	Total Estimasi
Backend Engineer	—	1.1, 1.3 (bareng GUI), 2.1, 2.2, 2.3, 2.4, 3.4	~19.5 jam
Algorithm Engineer	—	1.2	~3 jam
GUI Engineer	—	1.3 (bareng Backend), 3.1, 3.2, 3.3	~10.5 jam
QA / Tester	—	Verifikasi setiap fix, regression test	~4 jam
DevOps	—	Build & release setelah semua P1 selesai	~2 jam

8.5 Dampak yang Diharapkan Setelah Semua Fix

Metrik	Sebelum	Sesudah (estimasi)
Panggilan TraCI per step	~14.000+	~50–200
FPS GUI	8–12 (turun)	30 (stabil)
Thread safety	Crash acak	Stabil (shared buffer)
Memory (1 jam sim)	Terus bertambah	Stabil
Penyimpangan timing	Signifikan	< 1%

9. Cara Build & Deploy

9.1 Membuat Executable dengan PyInstaller

PyInstaller mengemas aplikasi Python + semua library + data jadi satu file executable. Mirip membuat file .exe di Windows.

```
# Install PyInstaller
pip install pyinstaller

# Build executable
pyinstaller tls.spec --clean -y

# Hasil:
#   Linux:   dist/tls/tls
#   Windows: dist/tls/tls.exe
#   macOS:  dist/TLS.app/
```

9.2 Isi File tls.spec

tls.spec adalah resep untuk PyInstaller. Isinya:

- **Analysis:** Daftar module Python yang perlu dibundel (traci, PyQt6, pyqtgraph, xml, json, csv, dll)
- **Datas:** Folder/data tambahan yang ikut dibundel (sim/ , resources/ , README.md)
- **Excludes:** Module yang tidak perlu (tkinter, matplotlib, OpenCV — menghemat ukuran)
- **console=True:** Menampilkan terminal di Windows (berguna untuk debug)
- **BUNDLE:** Khusus macOS — membuat .app bundle

9.3 Build Otomatis dengan GitHub Actions

File .github/workflows/build.yml mendefinisikan pipeline CI/CD yang berjalan otomatis:

Job	OS Runner	Aksi
test	ubuntu-latest	Install SUMO via apt, install Python deps, jalankan pytest
build-linux	ubuntu-latest	PyInstaller build + bundle SUMO binary + upload artifact
build-windows	windows-latest	PyInstaller build + download SUMO portable + bundle + upload artifact
release	ubuntu-latest	Download semua artifact, publish ke GitHub Releases

Pipeline ini berjalan saat:

- Push ke branch `main` → menjalankan test + build (tanpa release)
- Push tag `v*` (misal `v1.0.0`) → menjalankan test + build + release

9.4 Cara Membuat Release

```
# 1. Pastikan semua sudah di-commit dan di-push
git status
git push origin main

# 2. Buat tag versi baru
git tag -a v1.0.0 -m "Release v1.0.0"

# 3. Push tag ke GitHub
git push origin v1.0.0

# 4. Buka https://github.com/Cefneal/traffic-light-sim/releases
#   Tunggu ~5-10 menit, artifact akan muncul otomatis
```

9.5 Strategi Bundling SUMO

SUMO tidak dibundel ke dalam executable karena ukurannya terlalu besar (~200 MB) dan spesifik sistem operasi. Sebagai gantinya:

- **Linux:** GitHub Actions install SUMO via apt, lalu copy binary `sumo` ke folder bundle
- **Windows:** GitHub Actions download SUMO portable zip dari `sumo.dlr.de`, extract ke folder bundle
- **macOS:** Belum otomatis — pengguna harus install via `brew install sumo`

Saat aplikasi dijalankan, ia mencari SUMO dengan urutan: (1) PATH, (2) SUMO_HOME, (3) lokasi instalasi umum (Windows: `C:\Program Files\SUMO\bin\`).

10. Strategi Testing

10.1 Jenis Test

Level	Lingkup	Tools	Frekuensi
Unit Test	Fungsi individu (algoritma TL, logika caching)	pytest	Setiap commit
Integration Test	SimController + TraCIClient (end-to-end 1 step)	pytest + SUMO	Setiap PR
Performance Test	Hitung jumlah panggilan TraCI per step, FPS	pytest-benchmark	Mingguan
Regression Test	TL behavior, dashboard update, export	pytest (headless)	Setiap P1 fix

10.2 Rencana Test Cases

- **TestFixedTime:** TL berpindah fase pada interval yang benar
- **TestActuated:** Detector memicu perpanjangan fase hijau
- **TestMaxPressure:** Fase dipilih berdasarkan beban edge tertinggi
- **TestGreenWave:** Offset fase dihitung dengan benar
- **TestFuelMetrics:** Nilai fuel/CO2 dalam rentang yang wajar
- **TestThreadSafety:** Tidak crash setelah 1000 step dengan GUI membaca bersamaan
- **TestSubLeak:** Jumlah subscription kendaraan stabil setelah warmup

10.3 Regression Test untuk Setiap Fix P1

```
# 1. Smoke test: start sim, 100 step, stop
python -c "from tests.smoke import *; test_smoke()"

# 2. Bandingkan jumlah TraCI calls sebelum/sesudah
python -c "from tests.traci_count import *; compare_calls()"

# 3. Thread safety: loop 1000 step dengan GUI concurrent
python -c "from tests.thread_safety import *; test_no_crash(1000)"

# 4. Jalankan semua test
python -m pytest tests/ -v
```


11. Glosarium Istilah Lengkap

Istilah	Arti
Branch	Cabang kode yang terpisah dari main. Digunakan untuk mengembangkan fitur tanpa mengganggu kode utama.
CI/CD	Continuous Integration / Continuous Deployment. Sistem otomatis yang menjalankan test & build setiap kali kode di-push.
Clone	Mendownload seluruh riwayat proyek dari GitHub ke komputer lokal untuk pertama kali.
Commit	Merekam perubahan kode ke riwayat Git. Setiap commit punya pesan yang menjelaskan apa yang diubah.
Controller	Bagian dari MVC yang mengatur logika aplikasi (di sini: SimController).
Dashboard	Panel grafik real-time yang menampilkan metrik simulasi (kecepatan, throughput, dll).
Dependency	Library Python yang dibutuhkan aplikasi (tercantum di requirements.txt).
Detector / Induction Loop	Sensor di jalan yang mendeteksi kendaraan. Digunakan oleh algoritma Actuated.
Edge	Ruas jalan di SUMO. Setiap edge punya data: jumlah kendaraan, kecepatan rata-rata, dll.
Executable	File binary yang bisa dijalankan langsung tanpa perlu Python terinstall (misal: .exe di Windows).
Fase (Phase)	Status lampu lalu lintas pada suatu waktu, misal: "gggrrr" = 3 lajur hijau + 3 lajur merah.
FPS	Frames Per Second. Seberapa sering GUI memperbarui tampilan (target: 30).
GitHub Actions	Layanan CI/CD bawaan GitHub. Otomatis build & test saat push.
GUI	Graphical User Interface. Tampilan visual aplikasi (jendela, tombol, peta).
Import OSM	Mengunduh data peta dari OpenStreetMap dan mengonversinya ke format SUMO.
Merge	Menggabungkan perubahan dari satu branch ke branch lain.
Model	Bagian dari MVC yang mengelola data (di sini: TraCIClient).
MVC	Model-View-Controller. Pola arsitektur yang memisahkan data, logika, dan tampilan.
Pull	Mengambil perubahan terbaru dari GitHub ke komputer lokal.
Pull Request (PR)	Permintaan untuk menggabungkan kode dari branch fitur ke branch main. Biasanya direview dulu.
Push	Mengirim commit lokal ke GitHub.
PyInstaller	Tools untuk mengubah aplikasi Python jadi standalone executable.
Race Condition	Dua thread mengakses data yang sama bersamaan → hasil tidak terduga / crash.
Rebase	Memindahkan commit branch ke ujung branch lain. Alternatif merge yang menghasilkan riwayat lebih rapi.
Remote	Server Git jarak jauh (biasanya <code>origin</code> = GitHub).
Signal / PyQt6 Signal	Mekanisme komunikasi thread-safe di PyQt6. Sim thread kirim signal, GUI thread terima & proses.
Stash	Menyimpan perubahan sementara agar bisa pindah branch tanpa commit.
Step / Simulation Step	Satu iterasi simulasi (default: 1 detik simulasi).
Subscription	Fitur TraCI: berlangganan data tertentu (posisi kendaraan, status TL, dll) agar otomatis diterima setiap step tanpa diminta.
SUMO	Simulation of Urban MObility. Engine simulasi lalu lintas open-source dari DLR Jerman.
Tag	Penanda versi di Git. Biasanya untuk rilis (v1.0.0, v1.1.0, dll).
Thread	Unit eksekusi paralel. TLS punya 2 thread: sim (perhitungan) dan GUI (tampilan).
TL	Traffic Light. Lampu lalu lintas.
TraCI	Traffic Control Interface. Protokol TCP untuk mengontrol SUMO dari Python.
Venv	Virtual Environment. Lingkungan Python terisolasi agar library gak bentrok.
View	Bagian dari MVC yang menampilkan data ke pengguna (di sini: MapViewer, Dashboard).
Wheel	Format distribusi Python pre-built. Lebih cepat install daripada kompilasi dari source.

12. Lampiran: Referensi File Penting

12.1 Layer Engine (Model + Controller)

File	Isi Penting	Baris
app/engine/traci_client.py	TraCIClient: connect, subscribe, cached reads, TL control, fuel/CO2	499
app/engine/sim_controller.py	SimController: start, stop, _run_loop, step_single, emit signal	298
app/engine/tl_algorithms.py	4 algoritma: fixed, actuated, max_pressure, green_wave	173
app/engine/osm_importer.py	Konversi file .osm ke .net.xml via netconvert	~80

12.2 Layer GUI (View)

File	Isi Penting	Baris
app/gui/main_window.py	MainWindow: layout, menu bar, signal bridge, sim lifecycle	233
app/gui/map_viewer.py	MapView: render jaringan, kendaraan, TL, heatmap, tile OSM	674
app/gui/dashboard.py	DashboardPanel: pyqtgraph chart, export CSV/JSON	181
app/gui/controls.py	ControlsToolbar: play/pause/stop, speed, scenario dropdown	~150

12.3 Layer Data (Metrics + Models)

File	Isi Penting	Baris
app/metrics/collector.py	MetricsCollector: ring buffer, record, summary stats	83
app/metrics/storage.py	StorageManager: SQLite + JSON persistence	~130
app/models/traffic_light.py	TLPhase, TrafficLight: fase, durasi, elapsed time	~40
app/models/vehicle.py	Vehicle: posisi, kecepatan, sudut, tipe	~30

12.4 File Setup & Build

File	Fungsi
setup.bat	Setup untuk Windows CMD (create venv, install deps, cek SUMO)
setup.ps1	Setup untuk Windows PowerShell (sama, tapi lebih reliable)
setup.sh	Setup untuk Linux / macOS (detect brew, apt)
tls.spec	Resep PyInstaller: module, data, konfigurasi build
requirements.txt	Daftar dependency Python (PyQt6, pyqtgraph, traci, weasyprint)
.github/workflows/build.yml	Pipeline CI/CD: test → build Linux → build Windows → release
scripts/generate_docs_pdf.py	Script pembuat PDF dokumentasi ini

12.5 Struktur Folder sim/ (Data Map)

Map	Lokasi	Keterangan
Pamulang	sim/pamulang/	Kawasan Pamulang, Tangerang Selatan, Indonesia. ~15 TL.
Silicon Valley	sim/silicon_valley/	San Jose & sekitarnya, California, USA. ~77 TL.
Tokyo	sim/tokyo/	Shibuya & Shinjuku, Tokyo, Japan. ~100+ TL.

Setiap folder map berisi:

- *.net.xml — Jaringan jalan + TL (dihasilkan oleh netconvert)
- *.rou.xml — Rute kendaraan (kendaraan + perjalanan)
- *.sumocfg — Konfigurasi SUMO (menggabungkan .net.xml + .rou.xml)

- `buildings.json` — Data bangunan untuk tampilan visual (opsional)

— Akhir Dokumen —

Dibuat oleh generate_docs_pdf.py | TLS v1.0.0 | Bahasa Indonesia